

PLANT GUARDIAN

Technology Stack and Google Cloud Study Guide

A detailed learning guide connecting every major Plant Guardian feature to the frontend, backend, database, service, and Google Cloud technology that makes it work.

Item	Value
GCP project	gcp-hackathon-506604
Primary region	asia-south1 (Mumbai)
Production frontend	frontend Cloud Run service
Current frontend revision	frontend-00005-5d8
Core stack	Next.js, React, TypeScript, FastAPI, SQLAlchemy, PostgreSQL

Core product idea

Most trackers tell you good or bad. Plant Guardian tells you how urgent.

Prepared from the current Plant Guardian repository, deployment documentation, and deployed GCP architecture.

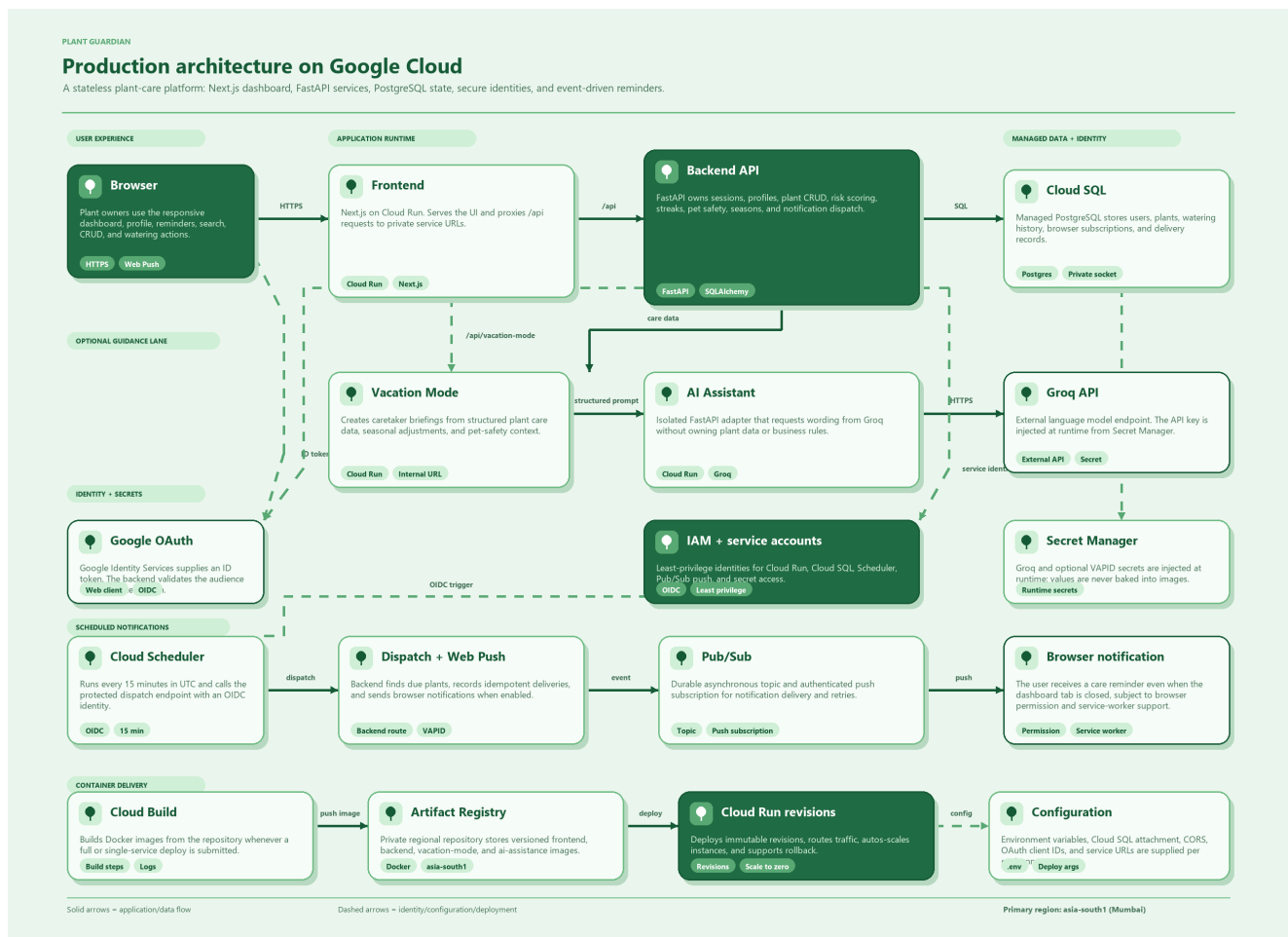
1. The complete system in one picture

The easiest mental model is: React and Next.js are the interface, FastAPI is the application brain, PostgreSQL is permanent memory, and GCP operates the production infrastructure.

Browser
 -> Next.js frontend
 -> FastAPI backend
 -> SQLAlchemy
 -> PostgreSQL

Vacation request
 -> Vacation Mode -> AI Assistance -> Groq

Automatic reminder
 -> Cloud Scheduler -> Backend -> Pub/Sub -> Web Push -> Browser



Memory aid

Next.js is the screen. FastAPI is the brain. PostgreSQL is memory. Cloud Run is the managed computer. Scheduler is the clock. Pub/Sub is the delivery queue. IAM is the security guard. Secret Manager is the vault.

2. Frontend technology stack

2.1 Next.js

Next.js is the production web framework. It serves the dashboard, performs production optimization, and provides same-origin API rewrites. The browser requests paths such as `/api/plants` or `/api/vacation-mode`, while Next.js forwards them to the correct Cloud Run service.

Why the proxy matters

The browser normally sees one frontend origin. Backend and Vacation Mode addresses remain server-side, simplifying CORS and hiding service topology from ordinary browser links.

2.2 React

React supplies the interactive component model. It owns plant-card state, forms, dialogs, loading states, filters, toasts, watering updates, profile editing, and the floating growth companion. After Just Watered returns an updated plant, React replaces that record without reloading the page.

2.3 TypeScript

TypeScript gives the frontend strict contracts for Plant, UserProfile, CareEvent, payloads, status values, and response fields. It catches many mismatched-field and nullability mistakes before deployment.

2.4 Tailwind CSS

Tailwind defines the responsive grid, green theme, spacing, card surfaces, forms, focus states, status styling, and mobile behavior through reusable utility classes and CSS-variable design tokens.

2.5 GSAP

GSAP handles the three-stage plant-card flip with controlled timing and easing. It also supports animation cleanup and reduced-motion behavior.

2.6 Phosphor Icons and inline SVG

Phosphor provides consistent interface icons. The account plant companion is a React-generated inline SVG, allowing growth stage, leaves, flowers, pot face, and mood to change without downloading separate image files.

Frontend responsibility	Main technology
Page and production runtime	Next.js
Interactive components	React
Data contracts	TypeScript
Responsive visual system	Tailwind CSS
Card transitions	GSAP
Icons and plant character	Phosphor Icons and inline SVG

3. Backend and database technology

3.1 Python and FastAPI

Python implements the domain logic. FastAPI exposes REST endpoints, authentication dependencies, structured errors, health checks, internal notification routes, and development API documentation.

3.2 Pydantic

Pydantic validates requests and responses. It ensures required fields exist, watering frequency is positive, enum-like values are valid, and frontend responses match the documented contract. Invalid requests normally return HTTP 422.

3.3 SQLAlchemy

SQLAlchemy maps Python models to PostgreSQL tables and handles queries, transactions, relationships, ownership filtering, and cascade deletion.

3.4 Alembic

Alembic manages database schema versions. The backend container runs alembic upgrade head before FastAPI starts, keeping a populated Cloud SQL database compatible with the deployed code.

3.5 PostgreSQL

PostgreSQL stores permanent state: users, password hashes, sessions, profiles, plants, waterings, care events, account XP, push subscriptions, and notification delivery records.

Computed, not stored

Risk score, days until due, seasonal effective frequency, status, and avatar mood are calculated at response time. This prevents values from becoming stale.

```
Frontend request
  -> FastAPI route
    -> Pydantic validation
      -> service/business logic
        -> SQLAlchemy transaction
          -> PostgreSQL
```

4. Technology behind each product feature

Feature	Technology path
Dashboard and plant cards	Next.js, React, TypeScript, Tailwind
Create, edit, and delete	React form -> FastAPI -> Pydantic -> SQLAlchemy -> PostgreSQL
Just Watered	React API call -> FastAPI transaction -> watering event -> updated response
Risk Score	Pure Python risk calculation in the backend
Healthy / Soon / Overdue	Backend status mapping plus Tailwind visual treatment
Search, room filter, sorting	TypeScript query functions and React state
Three-face card	React component state and GSAP animation
Plant facts and care guide	Bundled curated plant catalogue served by the backend
Ideal watering frequency	Species catalogue match with a seven-day fallback
Seasonal Shift Mode	Pure Python seasonal factor service
Pet-safety intelligence	Curated JSON dataset, Python resolver, stored safety flags
Email/password login	FastAPI auth, password hashing, HttpOnly session cookie, PostgreSQL
Google login	Google Identity Services and backend ID-token verification
Private garden	Authenticated ownership queries using user IDs
Care history	Append-only PostgreSQL care-event records
Garden streak	Python account-wide weekly care calculation

Feature	Technology path
Growth avatar	Backend XP and mood plus React inline SVG
Vacation Mode	Dedicated FastAPI service
AI caretaker wording	AI Assistance FastAPI service and Groq
AI fallback	Deterministic Python message builder
Browser reminders	Push API, service worker, VAPID, FastAPI
Cloud reminders	Cloud Scheduler, Pub/Sub, OIDC, Web Push
Local environment	Docker and Docker Compose
Quality checks	Pytest, FastAPI TestClient, Vitest, ESLint, Next.js build

5. Core business services

5.1 Risk service

backend/app/services/risk.py calculates days since watered, days until due, risk score, and status. The risk calculation uses the effective seasonal frequency when supplied.

```
risk = min(100, days_since_watered / effective_frequency * 100)
```

Score	Status	Meaning
0-39	Healthy	Plant is comfortably inside its watering window.
40-69	Needs Water Soon	The due date is approaching.
70-100	Overdue / High Risk	The plant needs attention.

5.2 Seasonal service

backend/app/services/seasons.py determines the Indian tropical season from the current UTC month and converts the user-configured base frequency into an effective frequency.

Season	Months	Factor
Summer	March-June	0.75
Monsoon	July-September	1.25
Post-monsoon	October-November	1.0
Winter	December-February	1.4

5.3 Pet-safety service

backend/app/services/pet_safety.py resolves a species against a curated static dataset. It returns safe, mild, or toxic flags, cat and dog toxicity, and placement advice. Safety facts are not generated by an LLM.

5.4 Account streak service

backend/app/services/account_streaks.py implements the garden-wide streak. At least 70 percent of the garden must receive care within the weekly window, one plant alone cannot advance it, and an overdue plant can break it.

5.5 Plant service

backend/app/services/plants.py coordinates plant queries, ownership, care calculations, watering, event creation, catalogue defaults, pet safety, seasonal context, and API response assembly.

6. AI services, local containers, and testing

6.1 Vacation Mode

The `vacation_mode` FastAPI service receives dates and selected plants, builds a deterministic watering schedule, applies seasonal cadence and pet warnings, and requests optional caretaker wording from AI Assistance.

6.2 AI Assistance

The `ai_assistance` FastAPI service is an isolated adapter around Groq. It receives structured care data, builds a controlled prompt, and returns natural-language instructions. It does not own risk, scheduling, streak, or pet-safety rules.

Reliability rule

If Groq is unavailable, Vacation Mode returns a deterministic fallback briefing. Core care decisions do not depend on the LLM.

6.3 Docker and Docker Compose

Docker packages each service with its runtime and dependencies. Docker Compose starts the frontend, backend, PostgreSQL, Vacation Mode, and AI Assistance together for local development.

```
docker compose up --build

Local services:
  frontend      :3000
  backend       :8000
  ai-assistance :8001
  vacation-mode :8002
  PostgreSQL    :5432
```

6.4 Testing

Backend tests use Pytest and FastAPI TestClient with an isolated database. Frontend tests use Vitest. ESLint checks code quality, and next build verifies TypeScript and the production bundle.

7. GCP service 1: Cloud Run

Cloud Run hosts Docker images as managed HTTPS services. It creates immutable revisions, routes traffic, emits logs and metrics, and scales instances according to demand.

Cloud Run service	Responsibility
frontend	Next.js dashboard and same-origin API proxy
backend	FastAPI auth, plants, care logic, database access, and notifications
vacation-mode	Vacation schedule and caretaker briefing composition
ai-assistance	Groq integration for natural-language care wording

Project use

The current frontend revision is `frontend-00005-5d8` and receives 100 percent of frontend traffic. Frontend-only changes can be deployed without touching the backend or database.

Why Cloud Run helps

- Automatic HTTPS
- Independent service deployment
- Revision history and rollback
- Request-based autoscaling

- Scale-to-zero support
- Integrated logs and metrics

Console: Cloud Run > Services

8. GCP services 2-4: data and container delivery

8.1 Cloud SQL

Cloud SQL runs the production PostgreSQL database. Instance plant-guardian-db contains database plant_guardian. The backend connects through the Cloud SQL Unix socket and uses the Cloud SQL Client role.

What it protects

User accounts, profiles, plants, watering history, care events, subscriptions, and delivery records survive container restarts and Cloud Run revision changes.

Console: SQL > Instances

8.2 Artifact Registry

Artifact Registry is the private Docker image warehouse. Repository plant-guardian stores versioned images for frontend, backend, vacation-mode, and ai-assistance.

```
asia-south1-docker.pkg.dev/gcp-hackathon-506604/plant-guardian/frontend:latest
```

Console: Artifact Registry > Repositories

8.3 Cloud Build

Cloud Build is the remote build factory. It receives source code, installs dependencies, runs the Dockerfile, builds the production application, and pushes the resulting image to Artifact Registry.

For the frontend, Cloud Build also injects API_URL, VACATION_API_URL, and NEXT_PUBLIC_GOOGLE_CLIENT_ID because these values are required during the Next.js production build.

Console: Cloud Build > History

9. GCP services 5-6: secrets and identity

9.1 Secret Manager

Secret Manager is the secure runtime vault. Plant Guardian stores the Groq API key and optional VAPID private key there. Values are injected into authorized Cloud Run services rather than being committed to Git or baked into images.

Not a secret

The Google OAuth Web client ID is public configuration. It must match the frontend and backend, but it does not need Secret Manager.

Console: Security > Secret Manager

9.2 IAM and service accounts

IAM applies least privilege. Each runtime or machine caller receives a dedicated identity with only the permissions needed for its job.

Identity	Attached to	Access
plant-guardian-frontend	Frontend Cloud Run	No project data roles
plant-guardian-backend	Backend Cloud Run	Cloud SQL, topic publisher, VAPID secret
plant-guardian-vacation	Vacation Mode	Service runtime only
plant-guardian-ai	AI Assistance	Groq secret accessor
plant-guardian-scheduler	Cloud Scheduler	Backend Cloud Run invoker
plant-guardian-pubsub-push	Pub/Sub push	Backend Cloud Run invoker

Security benefit

A service cannot automatically access every GCP resource. Scheduler and Pub/Sub call protected backend endpoints using Google-signed OIDC tokens instead of shared passwords.

Console: IAM & Admin > IAM and Service Accounts

10. GCP services 7-8: automatic reminders

10.1 Cloud Scheduler

Cloud Scheduler is the production clock. Job plant-guardian-notifications runs every 15 minutes and sends an authenticated POST request to /internal/notifications/dispatch.

The backend checks user reminder settings, local delivery time, plant due status, and whether that delivery has already been recorded.

Console: Cloud Scheduler > Jobs

10.2 Pub/Sub

Pub/Sub is the asynchronous delivery queue. The backend publishes a small delivery reference to topic plant-care-notifications. Push subscription plant-care-notifications-push calls the protected backend worker, which loads the full data from PostgreSQL and performs Web Push.

```

Cloud Scheduler
-> backend dispatch scan
    -> Pub/Sub topic
        -> authenticated push subscription
            -> backend delivery worker
                -> browser Web Push
  
```

Why Pub/Sub helps

- Decouples scanning from notification delivery
- Retries transient failures
- Allows Scheduler to finish quickly
- Supports idempotent delivery records
- Keeps plant and user data in PostgreSQL

Activation condition

The notification infrastructure is used when VAPID keys are configured. Without VAPID, the rest of Plant Guardian still deploys normally.

Console: Pub/Sub > Topics and Subscriptions

11. Google OAuth and application authentication

Google OAuth is configured through Google Auth Platform. It is an authentication integration rather than one of the eight infrastructure services above.

```
Continue with Google
-> Google Identity Services returns ID token
    -> frontend sends token to FastAPI
        -> backend verifies signature and audience
            -> backend creates or finds local account
                -> HttpOnly Plant Guardian session cookie
```

Configuration	Location	Purpose
NEXT_PUBLIC_GOOGLE_CLIENT_ID	Frontend build	Loads the correct Google Web client in the browser
GOOGLE_CLIENT_ID	Backend environment	Expected audience while verifying the ID token
Authorized JavaScript origins	Google Auth Platform	Permits localhost and the deployed frontend origin
Test users	Google Auth Platform	Allows selected accounts while the OAuth app is in Testing

Data ownership

Google proves the user's identity. Plant Guardian still owns the profile, pets, plants, watering history, XP, streak, and session data in PostgreSQL.

12. Local deployment versus GCP

Concern	Local development	GCP production
Application containers	Docker Compose	Cloud Run
Database	PostgreSQL container	Cloud SQL PostgreSQL
Image build	Local Docker build	Cloud Build
Image storage	Local Docker cache	Artifact Registry
Secrets	.env file	Secret Manager and Cloud Run configuration
Automatic schedule	Manual/local trigger	Cloud Scheduler
Notification queue	Optional direct delivery	Pub/Sub push workflow
Service identity	Local environment	IAM service accounts and OIDC

Deployment sequence

```
Source code
-> Cloud Build
    -> Docker image
        -> Artifact Registry
            -> Cloud Run revision
                -> 100 percent traffic after verification
```

Portability

Docker keeps the application architecture consistent. Local containers are replaced by managed GCP runtimes without rewriting the core application.

13. Interview-ready explanation

Plant Guardian is a containerized full-stack application built with Next.js, React and TypeScript on the frontend, and Python FastAPI, Pydantic and SQLAlchemy on the backend. PostgreSQL stores users, plants and watering history, while risk, seasonal frequency, pet safety and streak metrics are calculated by modular backend services. The system is deployed to four Cloud Run services, uses Cloud SQL for persistence, Cloud Build and Artifact Registry for container delivery, Secret Manager and IAM for security, and Cloud Scheduler with Pub/Sub for reliable browser reminders. Google OAuth provides optional sign-in, while Groq is isolated to non-critical caretaker wording.

Key design decisions to remember

- Core care calculations live on the backend, not in React.
- Calculated metrics are returned dynamically instead of stored stale.
- Safety facts come from curated static data, not AI.
- AI wording has a deterministic fallback.
- Each user has a private garden and account-level companion.
- Cloud services use dedicated least-privilege identities.
- Frontend, backend, Vacation Mode, and AI Assistance can be deployed independently.

One-line summary

Plant Guardian combines a typed modern frontend, a modular Python API, durable PostgreSQL state, and managed event-driven GCP infrastructure to tell users not only whether a plant needs water, but how urgent that care is.

Common interview questions

Question	Strong answer
What is unique?	A backend-calculated urgency score compares every plant on one 0-100 scale instead of showing only a due date.
Why not store risk score?	It depends on the current date and seasonal frequency. Calculating it on each response prevents stale data.
Why separate AI services?	Vacation scheduling remains deterministic, while optional LLM wording is isolated and can fail without breaking core care.
Why use Pub/Sub?	It separates reminder scanning from delivery, supports retries, and keeps Scheduler requests short.
How is security handled?	HttpOnly sessions, input validation, Secret Manager, dedicated IAM identities, OIDC callers, and private Cloud SQL access.
How can it scale?	Stateless Cloud Run services scale independently while shared durable state remains in managed PostgreSQL.